Chapter 3 Cleaning Code for Health Data

This chapter walks you through all of the R code used to clean the raw **HEALTH**-related data inputs. The resulting cleaned datasets are then used in the modeling (see next Chapter).

3.1 Create Final Data Inputs

This script imports all of raw health-related data inputs needed for the model.

Note that you must first open the 'methods_manual' R project before running this script or else it will not work.

First, let's set up our environment.

```
rm(list = ls())

options(scipen=999)

library(tidyverse)
library(readxl)
library(stringr)

# check working directory
getwd()
```

Create a date string that will be appended to all data file names.

```
my_date <- paste0("_", Sys.Date(), "_FINAL.csv")
```

3.1.1 Other Data Inputs

U.S. population

Just need to round the population number the nearest whole number. Then export data to FINAL folder.

```
pop <- read_xlsx("data_inputs/OTHER/us_population/DATA/NHANES 17 SUBPOPratio_0327.

pop1 <- pop %>% rename(subgroup = subgroup_id,
                       `2018_pop` = Pop_2018) %>%
  mutate(`2018_pop` = ceiling(`2018_pop`)) # round up to next whole number

write_csv(pop1, paste0("data_inputs/FINAL/cleaned_raw_data/population_distributior
```

Conversion units

Just select the needed variables then export.

```
units <- read_csv("data_inputs/OTHER/unit_conversions/DATA/Unit_conversions_1.4.24

units1 <- units %>% select(Food_group, DGA_unit, Conversion_to_grams, Equation)

write_csv(units1, paste0("data_inputs/FINAL/cleaned_raw_data/unit_conversions", my
```

Clear global environment expect for my_date.

```
rm(list=setdiff(ls(), c("my_date")))
```

3.1.2 Health Data Inputs

Cancer incidence

Import the raw cancer data input, clean up the variable names, and calculate the standard error of the cancer counts ('count_se').

```
# import cancer data
cancer <- read_xls("data_inputs/HEALTH/cancer_incidence/DATA/2018CANCERRATE_0327.x

cancer1 <-
  cancer %>% select(subgroup_id, cancer_code_ICD_O_3, Crude_Rate,
                    Standard_Error, `_2018_pop`, No_2018) %>%
  rename(crude_rate = Crude_Rate,
                    crude_se = Standard_Error,
                    count = No_2018,
                    population = `_2018_pop`,
           subgroup = subgroup_id,
           diseases = cancer_code_ICD_O_3) %>%
  mutate(count_se = (crude_se / 100000) * population)
```

Recode the cancer labels as shorter abbreviations.

```
cancer2 <- cancer1 %>% mutate(diseases = recode(diseases,
                                                "All Sites" = "ALL",
                                                "Colon and Rectum" = "CC",
                                                "Corpus Uteri" = "UC",
                                                "Esophagus" = "ECA",
                                                "Breast" = "BC",
                                                "Gallbladder" = "GC",
                                                "Kidney and Renal Pelvis" = "KC",
                                                "Liver and Intrahepatic Bile Duct" = "LVC",
                                                "Lung and Bronchus" = "LC",
                                                "myeloma" = "MMC",
                                                "Ovary" = "OC",
                                                "Pancreas" = "PC",
                                                "prostate(advanced)" = "APCA",
                                                "stomach_cardia" = "SCC",
                                                "stomach_noncar" = "SCNC",
                                                "Thyroid" = "TC",
                                                "oral cavity and pharynx and larynx" = "MLPC"
```

Round the count values to the nearest whole number.

```
cancer3 <- cancer2 %>% mutate(population = ceiling(population),
                              count = ceiling(count),
                              count_se = ceiling(count_se)) %>%
  arrange(diseases, subgroup)
```

Lastly, merge with population subsgroup information and cancer labels.

```
# import subgroups info
subgrps <- read_csv("data_inputs/OTHER/labels/DATA/population_subgroups_48_060923_

# join with cancer data
cancer4 <- left_join(cancer3, subgrps, by = "subgroup")

# import cancer label data
dis_labels <- read_csv("data_inputs/OTHER/labels/DATA/disease_outcomes_060923_FINA
  select(outcome, outcome_label)

cancer5 <- left_join(cancer4, dis_labels, by = c("diseases" = "outcome")) %>%
  rename(diseases_label = outcome_label) %>%
  relocate(diseases_label, .after = diseases)
```

If the rate or standard error variable are missing, then replace with 0. Lastly, export the cleaned file to the “FINAL” folder.

```
cancer6 <- cancer5 %>% mutate(crude_rate = ifelse(is.na(crude_rate), 0, crude_rate
                              crude_se = ifelse(is.na(crude_se), 0, crude_se),
                              count_se = ifelse(is.na(count_se), 0, count_se))

# export final dataset
write_csv(cancer6, paste0("data_inputs/FINAL/cleaned_raw_data/cancer_incidence", m
```

Cardiovascular disease (CVD) mortaliaty

Import raw CVD data and tidy.

```
cvd <- read_xlsx("data_inputs/HEALTH/cvd_mortality/DATA/CVDmotality2018_11022023.x

cvd1 <- cvd %>% select(cause, subgroup_id, Deaths, Deaths_se) %>%
  rename(subgroup = subgroup_id) %>%
  mutate(deaths_rounded = ceiling(Deaths),
         deaths_se_rounded = ceiling(Deaths_se))

# look at outcome labels
unique(cvd1$cause)

# need to rename diabetes
cvd2 <- cvd1 %>% mutate(cause = recode(cause,
                                "DM" = "DIAB")) %>%
  select(-c(Deaths, Deaths_se))
```

Transform the mean and SE data from long to wide format and then rejoin.

```
# just mean death values
cvd_wide_mean <- pivot_wider(cvd2 %>% select(-deaths_se_rounded),
                        names_from = cause,
                        values_from = deaths_rounded)

# just se death values
cvd_wide_se <- pivot_wider(cvd2 %>% select(-deaths_rounded),
                           names_from = cause,
                           values_from = deaths_se_rounded,
                           names_prefix = "se_")

# merge
cvd_wide <- left_join(cvd_wide_mean, cvd_wide_se, by = "subgroup")
```

Join with population subgroup information.

```r
# get rid of NA row
cvd_wide1 <- cvd_wide %>% filter(!(is.na(subgroup)))

# merge with cvd data
cvd_final <- left_join(cvd_wide1, subgrps, by = "subgroup") %>%
  relocate(subgroup, Age, Age_label, Sex, Sex_label, Race, Race_label)
```

Create the BMI-mediated and SBP-mediated variables by simply setting them equal to the cvd death values. This is needed for the model code to work properly later.

```
cvd_final1 <- cvd_final %>% mutate(

                        # BMI

                        AA_medBMI = AA,
                        se_AA_medBMI = se_AA,

                        AFF_medBMI = AFF,
                        se_AFF_medBMI = se_AFF,

                        CM_medBMI = CM,
                        se_CM_medBMI = se_CM,

                        DIAB_medBMI = DIAB,
                        se_DIAB_medBMI = se_DIAB,

                        ENDO_medBMI = ENDO,
                        se_ENDO_medBMI = se_ENDO,

                        HHD_medBMI = HHD,
                        se_HHD_medBMI = se_HHD,

                        HSTK_medBMI = HSTK,
                        se_HSTK_medBMI = se_HSTK,

                        IHD_medBMI = IHD,
                        se_IHD_medBMI = se_IHD,

                        ISTK_medBMI = ISTK,
                        se_ISTK_medBMI = se_ISTK,

                        OSTK_medBMI = OSTK,
                        se_OSTK_medBMI = se_OSTK,

                        OTH_medBMI = OTH,
```

```
  se_OTH_medBMI = se_OTH,

  PVD_medBMI = PVD,
  se_PVD_medBMI = se_PVD,

  RHD_medBMI = RHD,
  se_RHD_medBMI = se_RHD,

  TSTK_medBMI = TSTK,
  se_TSTK_medBMI = se_TSTK,

  # SBP

  AA_medSBP = AA,
  se_AA_medSBP = se_AA,

  AFF_medSBP = AFF,
  se_AFF_medSBP = se_AFF,

  CM_medSBP = CM,
  se_CM_medSBP = se_CM,

  DIAB_medSBP = DIAB,
  se_DIAB_medSBP = se_DIAB,

  ENDO_medSBP = ENDO,
  se_ENDO_medSBP = se_ENDO,

  HHD_medSBP = HHD,
  se_HHD_medSBP = se_HHD,

  HSTK_medSBP = HSTK,
  se_HSTK_medSBP = se_HSTK,

  IHD_medSBP = IHD,
  se_IHD_medSBP = se_IHD,
```

```
  ISTK_medSBP = ISTK,
  se_ISTK_medSBP = se_ISTK,

  OSTK_medSBP = OSTK,
  se_OSTK_medSBP = se_OSTK,

  OTH_medSBP = OTH,
  se_OTH_medSBP = se_OTH,

  PVD_medSBP = PVD,
  se_PVD_medSBP = se_PVD,

  RHD_medSBP = RHD,
  se_RHD_medSBP = se_RHD,

  TSTK_medSBP = TSTK,
  se_TSTK_medSBP = se_TSTK)
```

Export to FINAL folder.

```
write_csv(cvd_final1, paste0("data_inputs/FINAL/cleaned_raw_data/cvd_mortality", m
```

Other health data

Import other health datsets.

```
weight <- read_xls("data_inputs/HEALTH/overweight_prevalence/DATA/overweight1518_4

hbp <- read_xls("data_inputs/HEALTH/systolic_blood_pressure/DATA/SBP_1718_0419202

sbp <- read_xls("data_inputs/HEALTH/systolic_blood_pressure/DATA/SBP_1718_0419202

#high sbp variables
highSBP <- read_xlsx("data_inputs/HEALTH/systolic_blood_pressure/DATA/highSBP_data
```

Create non-Hipsanic Black (NHB) variables.

```
oth_health <- weight %>% mutate(nhb = ifelse(Race_label == "NHB", 1, 0),
                                nhb_se = 0) %>%
  rename(overweight_rate = Percent,
         overweight_rate_se = StdErr) %>%
  select(subgroup, overweight_rate, overweight_rate_se, nhb, nhb_se)
```

The SBP data is at the participant-level so it needs to be summarized at population subgroup level.

```r
sbp1 <- sbp %>%
  filter(!(is.na(mean_sbp))) # only include those with non-missing mean_sbp value

# summarize by subgroup ID
sbp_vars <- sbp1 %>%
  group_by(subgroup_id) %>%
  summarise(sbp_mean = mean(mean_sbp),
            # calculate standard error
            sbp_se = sd(mean_sbp)/sqrt(length(mean_sbp)))

# merge with highSBP
highSBP1 <- highSBP %>%
  select(subgroup_id, Percent, StdErr) %>%
  rename(highSBP_rate = Percent,
         highSBP_rate_se = StdErr)

sbp_vars1 <- left_join(sbp_vars, highSBP1, by = "subgroup_id")

# merge sbp vars with oth_health
oth_health1 <- left_join(oth_health, sbp_vars1, by = c("subgroup" = "subgroup_id")

# hbp variables
hbp1 <-
  hbp %>% rename(hbp = Percent,
                 hbp_se = StdErr) %>%
  mutate(hbp = hbp/100,
         hbp_se = hbp_se/100) %>%
  select(subgroup_id, hbp, hbp_se)

# merge hbp vars with oth_health
oth_health2 <- left_join(oth_health1, hbp1, by = c("subgroup" = "subgroup_id"))
```

Export to FINAL folder.

```
write_csv(oth_health2, paste0("data_inputs/FINAL/cleaned_raw_data/other_health", m
```

Effect sizes for diet and body mass index (BMI)

No changes are needed so just import the file then export to FINAL folder.

```
bmi_effects <- read_csv("data_inputs/HEALTH/effect_sizes_dietfactor_bmi/DATA/food_
```

```
write_csv(bmi_effects, paste0("data_inputs/FINAL/cleaned_raw_data/food_to_bmi_effe
```

Effect sizes for diet and systolic blood pressure (SBP)

No changes are needed so just import the file then export to FINAL folder.

```
sbp_effects <- read_csv("data_inputs/HEALTH/effect_sizes_dietfactor_sbp/DATA/food_
```

```
write_csv(sbp_effects, paste0("data_inputs/FINAL/cleaned_raw_data/food_to_sbp_effe
```

Relative risks (RR) for BMI and cancer

No changes are needed so just import the file then export to FINAL folder.

```
bmi_cancer <- read_csv("data_inputs/HEALTH/rr_bmi_cancer/DATA/RR_BMI_cancer_11.23.
```

```
write_csv(bmi_cancer, paste0("data_inputs/FINAL/cleaned_raw_data/rr_bmi_cancer", m
```

Relative risks (RR) for BMI and CVD

No changes are needed so just import the file then export to FINAL folder.

```
bmi_cvd <- read_csv("data_inputs/HEALTH/rr_bmi_cvd/DATA/RR_BMI_cvd_3.2.23.csv")

write_csv(bmi_cvd, paste0("data_inputs/FINAL/cleaned_raw_data/rr_bmi_cvd", my_date
```

Relative risks (RR) for SBP and CVD

No changes are needed so just import the file then export to FINAL folder.

```
sbp_cvd <- read_csv("data_inputs/HEALTH/rr_sbp_cvd/DATA/RR_sbp_cvd_2.16.23.csv")

write_csv(sbp_cvd, paste0("data_inputs/FINAL/cleaned_raw_data/rr_sbp_cvd", my_date
```

Log relative risks (LogRR) for diet-CVD

No changes are needed so just import the file then export to FINAL folder.

```
rm(list=ls(pattern="^cvd"))

cvd <- read_csv("data_inputs/HEALTH/logrr_dietfactor_disease/DATA/logRR_diet_cvd_b

write_csv(cvd, paste0("data_inputs/FINAL/cleaned_raw_data/logRR_diet_cvd", my_date
```

Log relative risks (LogRR) for diet-cancer

No changes are needed so just import the file then export to FINAL folder.

```
rm(list=ls(pattern="^cancer"))

cancer <- read_csv("data_inputs/HEALTH/logrr_dietfactor_disease/DATA/logRR_diet_ca

write_csv(cancer, paste0("data_inputs/FINAL/cleaned_raw_data/logRR_diet_cancer", m
```

TMRED

Import the TMRED values (in grams). Then, import the conversion units and join with the TMRED data.

```
tmred_g <- read_csv("data_inputs/HEALTH/tmred/DATA/TMRED_grams_1.5.24.csv")

# import conversion units
conversion <- read_csv(paste0("data_inputs/FINAL/cleaned_raw_data/unit_conversions

# join with tmred dataset
tmred_dga <- left_join(tmred_g, conversion, by = c("Risk_factor" = "Food_group"))
```

Then, transform the TMRED mean and standard deviation values from grams to “FPED units” (i.e., cups or ounces).

```
tmred_dga1 <- tmred_dga %>% mutate(tmred_dga_units = TMRED / Conversion_to_grams,
                      sd_dga_units = SD / Conversion_to_grams)

tmred_dga2 <- tmred_dga1 %>%
  select(Risk_factor, tmred_dga_units, sd_dga_units, DGA_unit) %>%
  rename(TMRED = tmred_dga_units,
         SD = sd_dga_units,
         Unit = DGA_unit) %>%
  mutate(TMRED = signif(TMRED, 3), # round to 3 sig figs
         SD = signif(SD, 3), # round to 3 sig figs
    Unit = str_sub(Unit, start = 3)) # fix unit var
```

Then, export the conversion units in both grams and FPED units to be used later.

```
# export grams dataset
write_csv(tmred_g, paste0("data_inputs/FINAL/cleaned_raw_data/tmred_grams", my_dat

# export FPED units
write_csv(tmred_dga2, paste0("data_inputs/FINAL/cleaned_raw_data/tmred_dga_units",
```

Clear global environment expect for my_date.

```
rm(list=setdiff(ls(), c("my_date")))
```

3.1.3 Diet Data Inputs

Dietary intake

Import the NHANES diet intake dataset and clean up.

```
nhanes <- read_csv("data_inputs/DIET/dietary_intake/DATA/output_data_from_cluster/

nhanes1 <- nhanes %>%
  rename("Foodgroup" = "food",
         "Mean_Intake" = "mean",
         "SE_Intake" = "SE",
         "Food_label" = "food_label",
         "Food_desc" = "food_desc") %>%
  mutate(Food_label = ifelse(Foodgroup == "kcal", "Kilocalorires", Food_label),
         Food_desc = ifelse(Foodgroup == "kcal", "Energy (kcal)", Food_desc)) %>%
  select(-c(starts_with(c("gro_", "oth_"))))
```

Then, convert the sugar sweetened beverage (SSB) mean and standard error from grams to cup (8 fl oz).

```
nhanes2 <- nhanes1 %>%
  # Divide by 240 to go from grams to cup
  mutate(Mean_Intake = ifelse(Foodgroup == "ssb", Mean_Intake/240, Mean_Intake),
  SE_Intake = ifelse(Foodgroup == "ssb", SE_Intake/240, SE_Intake),
  Food_desc = ifelse(Foodgroup == "ssb", "Sugar sweetened beverages (1 cup [8 fl c

nhanes3 <- nhanes2 %>% arrange(Foodgroup, subgroup)
```

Lastly, rename the standard deviation variable and then export.

```
# rename stddev
nhanes4 <- nhanes3 %>%
  rename(sigma_u_wgt = StdDev) %>%
  mutate(sigma_u_wgt = ifelse(Foodgroup == "ssb", sigma_u_wgt/240, sigma_u_wgt)) %
  relocate(sigma_u_wgt, .after = "SE_Intake")

# export
write_csv(nhanes4, paste0("data_inputs/FINAL/cleaned_raw_data/nhanes1518_agesexrac
```

Counterfactual intake

No changes are needed so just import the file then export to FINAL folder.

```
cf <- read_csv("data_inputs/DIET/counterfactual_intake/DATA/counterfactual_intake_

write_csv(cf, paste0("data_inputs/FINAL/cleaned_raw_data/counterfactual_intake", m
```

3.2 Clean Health Datasets

This script cleans all of the health-related data inputs needed for the model.

Note that you must first open the 'methods_manual' R project before running this script or else it will not work.

First, let's set up our environment.

```
rm(list = ls())

options(scipen=999)

library(tidyverse)
library(readxl)
library(stringr)

# check working directory
getwd()
```

Create a date string that will be appended to all data file names.

```
my_date <- paste0("_", Sys.Date(), "_FINAL.csv")
```

There are 13 health-related datasets that we will clean.

3.2.1 (1) Cancer incidence and (2) CVD mortality

In the code below, these two datasets get restructured and merged together into one "disease" dataset.

```r
# import cvd mortality dataset
cvd <- read_csv(paste0("data_inputs/FINAL/cleaned_raw_data/cvd_mortality", my_date
cancer <- read_csv(paste0("data_inputs/FINAL/cleaned_raw_data/cancer_incidence", m

# transform to long format (without se for now)
cvd_long_mean <- cvd %>%
  select(-starts_with("se_")) %>%
  pivot_longer(cols = !(c(subgroup, Age, Age_label, Sex, Sex_label, Race, Race_lab
               names_to = "diseases",
               values_to = "count")

# just do se now
cvd_long_se <- cvd %>%
  select(subgroup, Age, Age_label, Sex, Sex_label, Race, Race_label, starts_with("
  pivot_longer(cols = !(c(subgroup, Age, Age_label, Sex, Sex_label, Race, Race_lab
               names_to = "diseases",
               values_to = "count_se") %>%
  mutate(diseases = gsub("se_", "", diseases))

# combine
cvd_long <- left_join(cvd_long_mean,
                      cvd_long_se,
                      by = c("subgroup", "Age", "Age_label", "Sex", "Sex_label", "

# any missing?
cvd_long %>% filter(is.na(count) | is.na(count_se)) #none-good

# remove some vars from cancer dataset
cancer1 <- cancer %>%
  select(-c(diseases_label,
            # Sex_label, Age_label, Race_label,
            crude_rate, population, crude_se))

# bind cvd and cancer datasets together
cvd_cancer_merged <- rbind(cvd_long, cancer1) %>% rename(crude_se = count_se)
```

```
cvd_cancer_merged1 <- cvd_cancer_merged %>% arrange(diseases, subgroup)

# read in population distribution
pop <- read_csv(paste0("data_inputs/FINAL/cleaned_raw_data/population_distributior

cvd_cancer_merged2 <- left_join(cvd_cancer_merged1, pop, by = "subgroup") %>%
  rename(population = `2018_pop`)

# export merged dataset
write_csv(cvd_cancer_merged2, paste0("data_inputs/FINAL/model_data/cvd_cancer_merg
```

Then, we continue cleaning this merged disease dataset that is later appended to the NHANES dataset.

```r
# FYI - this function is stored in the LASTING Github repo
# This function was written by Fred
source("/Users/bmb73/Documents/GitHub/LASTING/Code/swap.r")

raw_file <- cvd_cancer_merged2

# remove all cancers and total stroke outcome categories
raw_file1 <- raw_file %>%
  # remove all cancers and total stroke outcome categories
  filter(diseases != "ALL" & diseases != "TSTK") %>%
  # remove BMImed and SBPmed for now
  filter(!(str_detect(diseases, "_medBMI|_medSBP"))) %>%
  # Fix Sex var
  mutate(Sex = factor(Sex),
         SexTemp = as.character(Sex),
         SexTemp = as.factor(SexTemp),
         # set SE
         se = crude_se) %>%
  # rename disease
  rename("disease" = "diseases")

# select subset of variables
mort <- raw_file1 %>%
  select(disease, subgroup, count, se) %>%
  rename(mn = count)

# transform long to wide
mort_wide <- pivot_wider(mort,
                         id_cols = subgroup,
                         names_from = disease,
                         values_from = c(mn, se),
                         names_glue = "{disease}{.value}")

# replace na with 0
mort_wide1 <- mort_wide %>%
```

```
  replace(is.na(.), 0)

# merge with subgroup info
subgrps <- read_csv("data_inputs/OTHER/labels/DATA/population_subgroups_48_060923_

# join
mort_wide2 <- mort_wide1 %>%
  left_join(subgrps, by = "subgroup")

mort_wide3 <- mort_wide2 %>%
  mutate(agecat = as.numeric(as.factor(Age_label)),
         female = ifelse(Sex_label == "Female", 1, 0),
         race = as.numeric(Race),
         racecat = race)

# rearrange variable order
mort_wide4 <- mort_wide3 %>%
  relocate(subgroup, agecat, female, race, racecat, Age, Age_label, Sex, Sex_label

# brooke fix this later

medBMI <- mort_wide4[, c(names(mort_wide4)[grep(pattern="mn", x=names(mort_wide4))
                         names(mort_wide4)[grep(pattern="se", x=names(mort_wide4))])]

names(medBMI)[grep(pattern="mn", names(medBMI))] <- gsub(pattern="mn", replacement
                                                         x=names(medBMI[grep(patterr

names(medBMI)[grep(pattern="se", names(medBMI))] <- gsub(pattern="se", replacement
                                                         x=names(medBMI[grep(patterr

medSBP <- mort_wide4[, c(names(mort_wide4)[grep(pattern="mn", x=names(mort_wide4))
                         names(mort_wide4)[grep(pattern="se", x=names(mort_wide4))])]

names(medSBP)[grep(pattern="mn", names(medSBP))] <- gsub(pattern="mn", replacement
                                                         x=names(medSBP[grep(patterr
```

```
names(medSBP)[grep(pattern="se", names(medSBP))] <- gsub(pattern="se", replacement
                                                         x=names(medSBP[grep(patterr

mort_wide5 <- cbind(mort_wide4, medBMI, medSBP)

# fix order again
mort_wide6 <- mort_wide5 %>%
  relocate(subgroup, agecat, female, race, racecat, Age, Age_label, Sex, Sex_label
  arrange(subgroup)

# export to FINAL folder
write_csv(mort_wide6, paste0("data_inputs/FINAL/cleaned_data/disease_incidence", m
```

3.2.2 (3) Effect sizes for diet and BMI and (4) diet and SBP

Clean up the global environment.

```
rm(list=setdiff(ls(), "my_date"))
```

First, import the effect sizes for diet and BMI.

```
ef <- read_csv(paste0("data_inputs/FINAL/cleaned_raw_data/food_to_bmi_effects" , m
```

We need to convert the effect sizes for four dietary factors: pf_pm, pf_redm, added_sugar, and leg_tot.

The current unit for pf_redm (red meat) is 3.5 oz eq/day, and needs to be transformed to 1 oz eq/day. So, we will divide the effects and standard errors by 3.5.

The current unit for pf_pm (processed meat) is 2 oz eq/day, and needs to be transformed to 1 oz eq/day. So, we will divide the effects and standard errors by 2.

The current unit for added_sugar is 1 gram/day, and needs to be transformed to 1 tsp/day. So, we will multiple the effects and standard errors by 4.2 (1 tsp=4.2 g).

The current unit for leg_tot is 1 cup/day, and needs to be transformed to 1 oz/day. So, we will multiple the effects and standard errors by (175/44).

```
ef_new <- ef %>%
  mutate(

    # effects for normal weight
    effect_normal_mean_converted = ifelse(food_group == "pf_redm" | food_group ==
                                   ifelse(food_group == "pf_pm", effect_normal_mea
                                   ifelse(food_group == "added_sugar", effect_norm
                                   ifelse(food_group == "leg_tot", effect_normal_m
                                          effect_normal_mean)))),

    effect_normal_mean_se_converted = ifelse(food_group == "pf_redm" | food_group
                                      ifelse(food_group == "pf_pm", effect_normal_
                                      ifelse(food_group == "added_sugar", effect_r
                                      ifelse(food_group == "leg_tot", effect_norma
                                             effect_normal_mean_se)))),

    # effects for overweight
    effect_overweight_mean_converted = ifelse(food_group == "pf_redm" | food_group
                                       ifelse(food_group == "pf_pm", effect_overwe
                                       ifelse(food_group == "added_sugar", effect_
                                       ifelse(food_group == "leg_tot", effect_over
                                              effect_overweight_mean)))),

    effect_overweight_mean_se_converted = ifelse(food_group == "pf_redm" | food_gr
                                          ifelse(food_group == "pf_pm", effect_ove
                                          ifelse(food_group == "added_sugar", effe
                                          ifelse(food_group == "leg_tot", effect_c
                                                 effect_overweight_mean_se))))

  ) %>%

  rename("effect_unit_converted" = nhanes_unit)

# look at original and converted values
ef_new_sub <- ef_new %>%
```

```
  select(food_group,
         effect_normal_mean, effect_normal_mean_converted,
         effect_normal_mean_se, effect_normal_mean_se_converted,
         effect_unit,
         effect_overweight_mean, effect_overweight_mean_converted,
         effect_overweight_mean_se, effect_overweight_mean_se_converted,
         effect_unit_converted)
```

Now, create the file that is used in the CRA model. Select relevant variables and rename variables.

```
ef_model_dat <- ef_new %>%
  select(food_group,
         effect_normal_mean_converted,
         effect_normal_mean_se_converted,
         effect_overweight_mean_converted,
         effect_overweight_mean_se_converted,
         effect_unit_converted) %>%
  rename("effect_normal_mean" = effect_normal_mean_converted,
         "effect_normal_mean_se" = effect_normal_mean_se_converted,
         "effect_overweight_mean" = effect_overweight_mean_converted,
         "effect_overweight_mean_se" = effect_overweight_mean_se_converted,
         "effect_unit" = effect_unit_converted)

# export to model folder
write_csv(ef_model_dat, paste0("data_inputs/FINAL/model_data/food_to_bmi_effects_c
```

Next, import the effect sizes for diet and SBP.

```
sbp_ef <- read_csv(paste0("data_inputs/FINAL/cleaned_raw_data/food_to_sbp_effects"
```

There is only one effect size that needs to be fixed: sodium.

The unit of the sodium-sbp effect size is currently mmHg/(1000mg/day). Therefore, the effect size needs to be divided by 1000 to get the unit mmHg/(1mg/day).

```
#divide columns 2-9 by 1000
idx <- c(2:ncol(sbp_ef))
sbp_ef[,idx] <- sbp_ef[,idx] / 1000

# Update the effect size unit label
sbp_ef_model_dat <- sbp_ef %>% mutate(effect_unit = ifelse(food_group == "sodium",

# export to TEMP folder
write_csv(sbp_ef_model_dat, paste0("data_inputs/FINAL/model_data/food_to_sbp_effec
```

3.2.3 (5) RRs for BMI and cancer and (6) for BMI and CVD

No unit conversions need to be made, so we just need to join the files and export.

```
rm(list=setdiff(ls(), "my_date"))

# import data
bmi_cancer <- read_csv(paste0("data_inputs/FINAL/cleaned_raw_data/rr_bmi_cancer",
bmi_cvd <- read_csv(paste0("data_inputs/FINAL/cleaned_raw_data/rr_bmi_cvd", my_dat

# join 2 bmi datasets
bmi_join <- left_join(bmi_cvd, bmi_cancer, by = c("risk_factor", "subgroup"))

# export to cleaned data folder
write_csv(bmi_join, paste0("data_inputs/FINAL/cleaned_data/rr_bmi_disease", my_dat
```

3.2.4 (7) LogRRs for diet and cancer and (8) for diet and

CVD

3.2.4.1 Part (i): Create LogRR dataset

The goal is to create a mega file that contains all diet and disease combinations (for all 48 subgroups).

```
rm(list=setdiff(ls(), "my_date"))
```

```r
# Retrieve list of disease outcomes
disease_dat <- read_csv("data_inputs/OTHER/labels/DATA/disease_outcomes_060923_FIN
disease_outcomes <- c(disease_dat$outcome)
disease_labels <- c(disease_dat$outcome_label)

# Retrieve list of dietary factors
diet_dat <- read_csv("data_inputs/OTHER/labels/DATA/dietary_factors_010424_FINAL.c
diet_factors <- c(diet_dat$Food_group)

# Retrieve population subgroups
pop_dat <- read_csv("data_inputs/OTHER/labels/DATA/population_subgroups_48_060923_

pop_dat1 <- pop_dat %>% select(subgroup, Age, Sex, Race)

# Number of population subgroups
subgroup_num <- 1:48

my_list <- list()

for (i in diet_factors) {

  dat <- tibble(
    #subgroup=1,
    outcome=disease_outcomes,
    #outcome_label=disease_labels,
    risk_factor=i)

  my_list[[i]] <- dat

}

dat_bind <- bind_rows(my_list)

#second loop
my_list1 <- list()
```

```r
for (j in subgroup_num) {

  dat_bind_more <-
    dat_bind %>%
    mutate(subgroup = j)

  my_list1[[j]] <- dat_bind_more

}

final_dat <- bind_rows(my_list1)

# Left join with pop dataset
final_dat1 <- final_dat %>% left_join(pop_dat1, by = "subgroup")

# JOIN WITH CVD AND CANCER DATASETS

# Retrieve cvd dataset
cvd_real <-
  read_csv(paste0("data_inputs/FINAL/cleaned_raw_data/logRR_diet_cvd", my_date)) %
  select(-c(outcome_label, Age_label))

# join with final_dat1
join1 <- left_join(final_dat1, cvd_real, by = c("outcome", "risk_factor", "Age"))

# test
join1 %>% filter(outcome == "OSTK" & risk_factor == "fruit_tot") %>% head() # look

# Retrieve cancer dataset
cancer_real <-
  read_csv(paste0("data_inputs/FINAL/cleaned_raw_data/logRR_diet_cancer", my_date)
  select(-c(outcome_label, RR, CI_lower, CI_upper, evidence)) %>%
  mutate(logRR_se = (logCI_upper - logCI_lower) / 3.92)

# join with join1
```

```r
join2 <- left_join(final_dat1, cancer_real, by = c("outcome", "risk_factor"))

# test
join2 %>% filter(outcome == "CC" & risk_factor == "dairy_tot") %>% head() #good
join2 %>% filter(outcome == "CC" & risk_factor == "dairy_cow") %>% head() #good
join2 %>% filter(outcome == "CC" & risk_factor == "dairy_soy") %>% head() #good
join2 %>% filter(outcome == "MLPC" & risk_factor == "fruit_tot") %>% head() #good

# need to merge join1 and join2

# get rid of rows where rr=na
join1_sub <- join1 %>% filter(!(is.na(logRR)))
join2_sub <- join2 %>% filter(!(is.na(logRR)))

# combine these datasets

# need to reconcile variable names
join2_sub <- join2_sub %>% select(-c(logCI_lower, logCI_upper))

my_join <- rbind(join1_sub, join2_sub)

# lastly, join with final_dat1 template
big_join <- left_join(final_dat1, my_join, by=NULL)

# retrieve conversion units for diet factors
convert <- read_csv(paste0("data_inputs/FINAL/cleaned_raw_data/unit_conversions",

# fill in NA values with logRR = 0
big_join1 <- big_join %>%
  mutate(logRR = ifelse(is.na(logRR), 0, logRR))

# merge with convert
big_join2 <- left_join(big_join1, convert, by = c("risk_factor" = "Food_group"))

# calculate RR unit if RR is missing
big_join3 <- big_join2 %>% mutate(RR_unit = ifelse(is.na(RR_unit),
```

```
                                    paste0(DGA_unit, "/day"),
                                    RR_unit))


# reorder columns
big_join4 <- big_join3 %>%
  relocate(subgroup, Age, Sex, Race) %>%
  arrange(subgroup, outcome)

# export
write_csv(big_join4,
          paste0("data_inputs/FINAL/cleaned_data/logRR_diet_disease", my_date))

# look at just the non-missing RRs
join_subset <- big_join4 %>% filter(logRR != 0)

# export
write_csv(join_subset,
          paste0("data_inputs/FINAL/cleaned_data/logRR_diet_disease_SUBSET", my_da
```

3.2.4.2 Part (ii): Convert LogRR Units

Now we need to convert the units of the LogRR dataset we just created.

```
rm(list=setdiff(ls(), "my_date"))
```

```r
# Read in diet-disease file
rr <- read_csv(paste0("data_inputs/FINAL/cleaned_data/logRR_diet_disease", my_date
               col_types = "ddddccddccdc") %>%
  select(-c(Age, Sex, Race))

# create variable that contains the numeric value of the original unit
rr <- rr %>%
  mutate(RR_unit_num = parse_number(RR_unit),
         unit_type = sub(".*/", "", RR_unit))

# fix "day"
rr <- rr %>% mutate(unit_type = ifelse(unit_type == "d", "day", unit_type))

table(rr$unit_type, useNA = "always")

# Function 1: Day function
# Use this function when we need to convert the RR unit from /week to /day

day_func <- function(x, y, u){

  # x=dataset
  # y=dietary factor name
  # u=newly converted unit

  x %>%
    split(list(x$risk_factor, x$unit_type)) %>%
    modify_at(paste0(y, ".wk"), ~mutate(.,
                                logRR = logRR * 7,
                                logRR_se = logRR_se * 7,
                                RR_unit = paste0(RR_unit_num, " ", u),
                                unit_type = "day")) %>%
    bind_rows()
}

# test
```

```
day_func(x=rr, y="pf_redm", u="g/day") %>% filter(risk_factor=="pf_redm" & logRR !

# apply to the RRs that are in week unit

rr %>% filter(unit_type == "wk") %>% select(risk_factor) %>% distinct()
# pf_ns, pf_redm, pf_pm, pf_seafood

rr1 <- rr %>%

  day_func(y="pf_ns", u="oz eq/day") %>%
  day_func(y="pf_redm", u="g/day") %>%
  day_func(y="pf_redm_tot", u="g/day") %>%
  day_func(y="pf_pm", u="g/day") %>%
  day_func(y="pf_seafood", u="g/day")


# check
rr1 %>% filter(risk_factor=="pf_seafood" & logRR != 0) %>% head() #good
rr1 %>% filter(risk_factor=="pf_redm_tot" & logRR != 0) %>% head() #good


# Function 2: Convert function
# Use this function when we need to convert the RR unit to a different type of uni

convert_func <- function(x, y){

  # x=dataset
  # y=dietary factor name
  # z=conversion factor
  # u=newly converted unit

  x %>% split(~risk_factor == paste0(y)) %>%
    modify_at("TRUE", ~mutate(.,
                              RR_converted = (logRR / RR_unit_num) * Conversion_tc
                              SE_converted = (logRR_se / RR_unit_num) * Conversior
                              RR_unit_converted = paste0(DGA_unit, "/", unit_type)
```

```
                                ))  %>%
    bind_rows()
}

# test
rr1 %>% convert_func(y="pf_poultry") %>% filter(risk_factor=="pf_poultry") %>% hea
rr1 %>% convert_func(y="dairy") %>% filter(risk_factor=="dairy") %>% head()
rr1 %>% convert_func(y="fruit_juice") %>% filter(risk_factor=="fruit_juice") %>% h


# APPLY FUNCTION TO DIET FACTORS

rr2 <- rr1 %>%

  # RRs exist
  # grams to cups or oz
  convert_func(y="dairy_tot") %>%
  convert_func(y="dairy_cow") %>%
  convert_func(y="dairy_soy") %>%
  convert_func(y="fruit_tot") %>%
  convert_func(y="fruit_exc_juice") %>%
  convert_func(y="veg_exc_sta") %>%
  convert_func(y="gr_whole") %>%
  convert_func(y="leg_tot") %>%
  convert_func(y="pf_pm") %>%
  convert_func(y="pf_redm") %>%
  convert_func(y="pf_redm_tot") %>%
  convert_func(y="pf_seafood") %>%

  # RRs don't exist
  convert_func(y="pf_poultry")  %>%
  convert_func(y="pf_poultry_tot")  %>%
  convert_func(y="pf_egg")  %>%
  convert_func(y="pf_leg") %>%
  convert_func(y="pf_soy") %>%
```

```
  convert_func(y="fruit_juice") %>%
  convert_func(y="veg_dg") %>%
  convert_func(y="veg_leg") %>%
  convert_func(y="veg_oth") %>%
  convert_func(y="veg_ro") %>%
  convert_func(y="veg_sta") %>%
  convert_func(y="added_sugar") %>%
  convert_func(y="gr_refined") %>%
  convert_func(y="oil") %>%
  convert_func(y="sodium") %>%

  # conversion factors don't exist
  # and we're not using these dietary factors for the CRA anyway
  convert_func(y="pf_animal") %>%
  convert_func(y="pf_plant") %>%
  convert_func(y="babyfood") %>%
  convert_func(y="coffee_tea") %>%
  convert_func(y="other") %>%
  convert_func(y="water")

# check
rr2 %>% filter(risk_factor=="leg_tot" & RR_converted !=0) %>% head()
rr2 %>% filter(risk_factor=="sodium") %>% head()
rr2 %>% filter(risk_factor=="water") %>% head()



# Function 3: Divide function
# Use this function when we need to scale down the RR unit (e.g., 100 mg to 1 mg)

divide_func <- function(x, y){

  # x=dataset
  # y=dietary factor name
  # u=newly converted unit

  x %>%
```

```
    split(~risk_factor == paste0(y)) %>%
    modify_at("TRUE", ~mutate(.,
                              RR_converted = logRR / RR_unit_num,
                              SE_converted = logRR_se / RR_unit_num,
                              RR_unit_converted = paste0(DGA_unit, "/", unit_type)
                              )) %>%
    bind_rows()
}

# APPLY

rr3 <- rr2 %>%

  # scale down units
  divide_func(y="sea_omega3_fa") %>% # want to get unit 1 mg/day
  divide_func(y="fiber") %>% # want to get unit 1 g/day
  divide_func(y="pufa_rep_carb") %>% # want to get unit 1%E/day
  divide_func(y="pufa_rep_sfa") %>%  # want to get unit 1%E/day
  divide_func(y="pufa_rep_carbsfa") %>%   # want to get unit 1%E/day
  divide_func(y="sat_fat") # want to get unit 1%E/day

# check
rr3 %>% filter(risk_factor=="sea_omega3_fa" & RR_converted !=0) %>% head()
rr3 %>% filter(risk_factor=="fiber" & RR_converted !=0) %>% head()



# Function 4: No changes needed
# Use this function when the unit is already correct

nochange_func <- function(x, y){

  # x=dataset
  # y=dietary factor name
  # u=newly converted unit

  x %>%
```

```
    split(~risk_factor == paste0(y)) %>%
    modify_at("TRUE", ~mutate(.,
                              RR_converted = logRR,
                              SE_converted = logRR_se ,
                              RR_unit_converted = paste0(DGA_unit, "/", unit_type)
    bind_rows()
}


# APPLY

rr4 <- rr3 %>%

  # no changes needed
  nochange_func(y="ssb") %>%
  nochange_func(y="pf_ns")

# check
rr4 %>% filter(risk_factor=="ssb" & RR_converted !=0) %>% head()
rr4 %>% filter(risk_factor=="pf_ns" & RR_converted !=0) %>% head()

# determine if any RRs have not been converted yet
rr4 %>% filter(is.na(RR_converted)) %>% select(risk_factor) %>% unique() #none-goo

# rearrange the data by subgroup and outcome
rr5 <- rr4 %>% arrange(subgroup, outcome)

# now create full dataset for the model

# need to rename variable names
rr6 <- rr5 %>%
  select(-c(RR_unit, logRR, logRR_se, Conversion_to_grams, RR_unit_num, unit_type,
  rename("RR" = RR_converted,
         "SE" = SE_converted,
         "RR_unit" = RR_unit_converted)
```

```
# if rr is 0, then set se=0
rr7 <- rr6 %>% mutate(SE = ifelse(RR == 0, 0, SE))

# export to TEMP folder
write_csv(rr7, paste0("data_inputs/FINAL/cleaned_data/logRR_diet_disease_converted
```

3.2.5 (9) RRs for SBP and CVD

```
rm(list=setdiff(ls(), "my_date"))
```

No conversions need to be made to the RRs for SBP and CVD, but we do need to merge and reformat all of the RRs into one file.

```r
raw.RRs <- read_csv(paste0("data_inputs/FINAL/cleaned_data/logRR_diet_disease_conv

# need to rename 2 vars
raw.RRs <- raw.RRs %>% rename("logRR" = RR,
                              "logRRse" = SE)

# remove TSTK rows
raw.RRs <- raw.RRs %>% filter(outcome != "TSTK")

n.diseases <- length(unique(raw.RRs$outcome))
vars.to.keep <- c("subgroup","risk_factor", "outcome", "logRR", "logRRse")

raw.RRs <- raw.RRs[,vars.to.keep]

# this is better way to do it
food.names <- unique(raw.RRs$risk_factor)

# at this point, all of the units are 1
RR_unit <- rep(1, length(food.names))

disease.names <- as.character(unique(raw.RRs$outcome))
disease.names.se <- paste(disease.names, "se", sep="")
col.names.main <- as.vector(rbind(disease.names, disease.names.se))

RRs.wide <- pivot_wider(raw.RRs,
            names_from = "outcome",
            values_from = c("logRR", "logRRse"),
            names_sep = ".")

RRs.wide[is.na(RRs.wide)] <- 0

# renaming to columns to make consistent with old names from CVD papers
names(RRs.wide)[grep(pattern="logRRse.", x=names(RRs.wide))] <-
  paste(gsub(pattern="logRRse.", replacement="", x=names(RRs.wide)[grep(pattern="l
        replacement="se",
```

```
        sep="")

names(RRs.wide) <- gsub(pattern="logRR.", replacement="", x=names(RRs.wide))
names(RRs.wide)[names(RRs.wide)=="risk_factor"] <- "RF"

RRs.wide$RRunit <- 0

for(i in 1:length(food.names))
{
  RRs.wide$RRunit[RRs.wide$RF==food.names[i]] <- RR_unit[i]
}

RRs.wide<-RRs.wide[order(RRs.wide$RF, RRs.wide$subgroup),]

# Now let's move on to mediated effects, that is effects of BMI on sex, and effect
disease.names.BMImed <- paste(disease.names, "_medBMI", sep="")

disease.names.BMImed.se <- paste(disease.names.BMImed, "se", sep="")

col.names.BMI <- as.vector(rbind(disease.names.BMImed, disease.names.BMImed.se))

BMI_RRs<-read.csv(paste0("data_inputs/FINAL/cleaned_data/rr_bmi_disease", my_date)

BMI_RRs <- BMI_RRs %>% select(-c(ALL, ALLse))

new.order <- c("risk_factor","subgroup", as.vector(rbind(disease.names, disease.na

BMI_RRs <- BMI_RRs[,new.order[new.order %in% names(BMI_RRs)]]

current.bmi.colnames <- names(BMI_RRs)[3:(length(names(BMI_RRs)))]

for(i in grep("se", current.bmi.colnames))
{
  names(BMI_RRs)[2+i] <- paste(strsplit(current.bmi.colnames[i], split="se"), "_me
  names(BMI_RRs)[2+i-1] <- paste(names(BMI_RRs)[2+i-1], "_medBMI", sep="")
}
```

```
BMI_RRs<-BMI_RRs[order(BMI_RRs$risk_factor,BMI_RRs$subgroup),]

# effects of SBP on disease
disease.names.SBPmed <- paste(disease.names, "_medSBP", sep="")

disease.names.SBPmed.se <- paste(disease.names.SBPmed, "se", sep="")

col.names.SBP <- as.vector(rbind(disease.names.SBPmed, disease.names.SBPmed.se))

# This dataset didn't need any changes
# so it's in the raw data folder
SBP_RRs <- read.csv(paste0("data_inputs/FINAL/cleaned_raw_data/rr_sbp_cvd", my_dat

new.order <- c("risk_factor","subgroup", as.vector(rbind(disease.names, disease.na

SBP_RRs <- SBP_RRs[,new.order[new.order %in% names(SBP_RRs)]]

current.sbp.colnames <- names(SBP_RRs)[3:(length(names(SBP_RRs)))]

for(i in grep("se", current.sbp.colnames))
{
  names(SBP_RRs)[2+i] <- paste(strsplit(current.sbp.colnames[i], split="se"), "_me
  names(SBP_RRs)[2+i-1] <- paste(names(SBP_RRs)[2+i-1], "_medSBP", sep="")
}

SBP_RRs <- SBP_RRs[order(SBP_RRs$risk_factor,SBP_RRs$subgroup),]

# remove risk factor var
SBP_RRs <- SBP_RRs %>% select(-risk_factor)
BMI_RRs <- BMI_RRs %>% select(-risk_factor)

allRRs <- Reduce(function(x, y) merge(x, y, by.x="subgroup"), list(RRs.wide, BMI_R
allRRs <- allRRs[order(allRRs$RF, allRRs$subgroup), ]
```

```
# export
write_csv(allRRs, paste0("data_inputs/FINAL/model_data/rr_agesexrace", my_date))
```

3.2.6 (10) Current intake (NHANES), (11) TMRED, (12) Counterfactual, and (13) Other health data

Other health data includes overweight rate, average SBP, % with hypertension, and high SBP rate.

The other health dataset does not need to be changed, but it needs to be merged with the NHANES dataset.

```
rm(list=setdiff(ls(), "my_date"))
```

```r
# import
nhanes.dat <- read_csv(paste0("data_inputs/FINAL/cleaned_raw_data/nhanes1518_agese

# rename var
nhanes.dat1 <- nhanes.dat %>% rename(diet = Foodgroup,
                                     mean = Mean_Intake,
                                     se = SE_Intake,
                                     #n = N,
                                     #intake_unit = IntakeUnit,
                                     diet_label = Food_label)

# sea_omega3_fa is in grams and we want it in milligrams

# first, look at data
nhanes.dat1 %>% filter(diet == "ssb") %>% head() #this is fine
nhanes.dat1 %>% filter(diet == "sea_omega3_fa") %>% head()

# fix sea_omega3_fa
# g -> mg
# 1 g = 1000 mg
nhanes.dat2 <- nhanes.dat1 %>%
  split(~diet == "sea_omega3_fa") %>%
  modify_at("TRUE", ~mutate(.,
                            mean = mean * 1000,
                            se = se * 1000,
                            sigma_u_wgt = sigma_u_wgt * 1000)) %>%
  bind_rows()

# look at data
# nhanes.dat2 %>% filter(diet == "ssb") %>% head()
nhanes.dat2 %>% filter(diet == "sea_omega3_fa") %>% head()

# read in counterfactual data
counterfactuals <- read_csv(paste0("data_inputs/FINAL/cleaned_raw_data/counterfact
```

```r
# rename vars
counterfactuals <- counterfactuals %>%
  rename(diet = food_group,
         CF_intake_unit = intake_units) %>%
  select(diet, diet_pattern, CF_mean_intake, CF_se_intake, CF_sd_intake, CF_intake

# manually fix some of the units
# added sugar (g -> tsp)
# pf_leg (cup -> oz)

counterfactuals1 <- counterfactuals %>%
  mutate(#added sugar

         CF_mean_intake = round(ifelse(diet == "added_sugar", CF_mean_intake / 4.2
         CF_sd_intake = round(ifelse(diet == "added_sugar", CF_sd_intake / 4.2, CF
         CF_intake_unit = ifelse(diet == "added_sugar", "tsp eq/day", CF_intake_ur

         # pf_leg

         CF_mean_intake = round(ifelse(diet == "pf_leg", CF_mean_intake * 4, CF_me
         CF_sd_intake = round(ifelse(diet == "pf_leg", CF_sd_intake * 4, CF_sd_int
         CF_intake_unit = ifelse(diet == "pf_leg", "ounce eq/day", CF_intake_unit)

# read in TMRED data
tmred <- read_csv(paste0("data_inputs/FINAL/cleaned_raw_data/tmred_dga_units", my_

# rename vars
tmred <- tmred %>% rename(TMRED_mean_intake = TMRED,
                  TMRED_sd_intake = SD,
                  TMRED_intake_unit = Unit)

# read in death data
deaths.dat <- read_csv(paste0("data_inputs/FINAL/cleaned_data/disease_incidence",

# read in other health data
other_healh <- read_csv(paste0("data_inputs/FINAL/cleaned_raw_data/other_health",
```

```r
# merge nhanes and counterfactual
dat1 <- left_join(nhanes.dat2, counterfactuals1, by = "diet")

# check pf_seafood to make sure all 5 dietary patterns were merged
dat1 %>% filter(diet == "pf_seafood") %>% select(subgroup, diet, diet_pattern, CF_
#looks good

# merge dat1 with tmred
dat2 <- left_join(dat1, tmred, by = c("diet" = "Risk_factor"))

# merge dat2 with other health data
dat3 <- left_join(dat2, other_healh, by = "subgroup")

# lastly, merge with deaths data
merged <- left_join(dat3, deaths.dat, by = "subgroup")

# create merged file
merged1 <- merged %>%
  arrange(diet, subgroup, diet_pattern) %>%
  relocate(c(agecat, Age_label, female, Sex, race, Race_label), .after = subgroup)

# check data
merged1 %>% select(subgroup, overweight_rate) %>% head()

# export file
write_csv(merged1, paste0("data_inputs/FINAL/model_data/nhanes1518_agesexrace_merg
```

3.3 Restructure data

This script transforms the NHANES dataset to match the structure of the cost/environment mega dataset.

```
rm(list=setdiff(ls(), "my_date"))


# IMPORT NHANES DATA

nhanes <- read_csv(paste0("data_inputs/FINAL/model_data/nhanes1518_agesexrace_merg
```

```
## Rows: 6480 Columns: 211
## ── Column specification ────────────────────────────────────────────────────
## Delimiter: ","
## chr   (9): Age_label, Race_label, diet, diet_label, Food_desc, diet_pattern, CF_intake_u
## dbl (202): subgroup, agecat, female, Sex, race, N, mean, se, sigma_u_wgt, pro_gro, CF_me
##
## ℹ Use `spec()` to retrieve the full column specification for this data.
## ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```r
# update variable names

nhanes1 <- nhanes %>% rename(subgroup_id = subgroup,
                  Foodgroup = diet,
                  Mean_Intake = mean,
                  SE_Intake = se,
                  CF_Mean_Intake = CF_mean_intake,
                  CF_SE_Intake = CF_se_intake,
                  TMRED_Mean_Intake = TMRED_mean_intake,
                  TMRED_SD_Intake = TMRED_sd_intake) %>%
  mutate(datatype="Total",
         pro_nongro = 1 - pro_gro) %>%
  relocate(datatype, .after = Food_desc) %>%
  relocate(pro_nongro, .after = pro_gro)

# remove extra ssb rows
nhanes1 %>% filter(Foodgroup == "ssb") %>% head()
```

```
## # A tibble: 6 × 213
##   subgroup_id agecat Age_label female   Sex  race Race_label Foodgroup     N Mean_Intake
##         <dbl>  <dbl> <chr>      <dbl> <dbl> <dbl> <chr>      <chr>     <dbl>       <dbl>
## 1           1      1 20-34          1     1     1 NHW        ssb         381        1.42
## 2           1      1 20-34          1     1     1 NHW        ssb         381        1.42
## 3           1      1 20-34          1     1     1 NHW        ssb         381        1.42
## 4           1      1 20-34          1     1     1 NHW        ssb         381        1.42
## 5           1      1 20-34          1     1     1 NHW        ssb         381        1.42
## 6           2      1 20-34          1     1     2 NHB        ssb         282        1.29
## # ℹ 201 more variables: diet_label <chr>, Food_desc <chr>, datatype <chr>, pro_gro <dbl>
## #   diet_pattern <chr>, CF_Mean_Intake <dbl>, CF_SE_Intake <dbl>, CF_sd_intake <dbl>, CF
## #   TMRED_Mean_Intake <dbl>, TMRED_SD_Intake <dbl>, TMRED_intake_unit <chr>, overweight_
## #   overweight_rate_se <dbl>, nhb <dbl>, nhb_se <dbl>, sbp_mean <dbl>, sbp_se <dbl>, hig
## #   highSBP_rate_se <dbl>, hbp <dbl>, hbp_se <dbl>, racecat <dbl>, Age <dbl>, Sex_label
## #   AFFmn <dbl>, APCAmn <dbl>, BCmn <dbl>, CCmn <dbl>, CMmn <dbl>, DIABmn <dbl>, ECAmn <
## #   HHDmn <dbl>, HSTKmn <dbl>, IHDmn <dbl>, ISTKmn <dbl>, KCmn <dbl>, LCmn <dbl>, LVCmn
```

```r
nhanes2 <- nhanes1 %>% filter(!(is.na(subgroup_id)))

# create grocery data
grocery <- nhanes2 %>% mutate(datatype = recode(datatype,
                                                `Total` = "Grocery"),
                              Mean_Intake = Mean_Intake * pro_gro,
                              SE_Intake = SE_Intake * pro_gro,
                              sigma_u_wgt = sigma_u_wgt * pro_gro,

                              CF_Mean_Intake = CF_Mean_Intake * pro_gro,
                              CF_SE_Intake = CF_SE_Intake * pro_gro,
                              CF_sd_intake = CF_sd_intake * pro_gro)


nongrocery <- nhanes2 %>% mutate(datatype = recode(datatype,
                                                   `Total` = "Non-Grocery"),
                                 Mean_Intake = Mean_Intake * pro_nongro,
                                 SE_Intake = SE_Intake * pro_nongro,
                                 sigma_u_wgt = sigma_u_wgt * pro_nongro,

                                 CF_Mean_Intake = CF_Mean_Intake * pro_nongro,
                                 CF_SE_Intake = CF_SE_Intake * pro_nongro,
                                 CF_sd_intake = CF_sd_intake * pro_nongro)


# bind together
nhanes_comb <- rbind(nhanes2, grocery, nongrocery)

# rearrange
nhanes_comb1 <-
  nhanes_comb %>%
  arrange(subgroup_id, Foodgroup, datatype, diet_pattern) %>%
  relocate(c(datatype, diet_pattern), .before = Mean_Intake)

# lastly, add 2018 population variable
```

```
# read in population distribution
pop <- read_csv(paste0("data_inputs/FINAL/cleaned_raw_data/population_distributior
```

```
## Rows: 48 Columns: 2
## ── Column specification ────────────────────────────────────────────────────────
## Delimiter: ","
## dbl (2): subgroup, 2018_pop
##
## ℹ Use `spec()` to retrieve the full column specification for this data.
## ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
nhanes_comb2 <- left_join(nhanes_comb1, pop, by = c("subgroup_id" = "subgroup")) %
  rename(population = `2018_pop`) %>%
  relocate(population, .before = overweight_rate)

# export
write_csv(nhanes_comb2, paste0("data_inputs/FINAL/cleaned_data/mega_costenv_struct
```